浏览器工作原理浅析

TID Ghostzhang

财付通设计中心
Tenpay Innovation Design

本PPT只是对内容的一个整理

+Ghost 搜索 图片 视频 地图 新闻 购物 Gmail 更多 Ghost Zhang 0 分享...

Google 浏览器工作原理

搜索 找到约 11,900,000 条结果 （用时 0.28 秒）

所有结果
图片
地图
视频
新闻
购物
更多

网页
所有中文网页
简体中文网页
翻译的外文网页

所有结果
图文并茂
更多搜索工具

从输入网址到显示页面：浏览器工作原理拆解分析(1) - 51CTO.COM
developer.51cto.com/art/201007/209634.htm - 网页快照
2010年7月6日 – 本文深入的研究了浏览器从输入网址到显示页面之间发生的事情。本文内容面向对网络应用相关技术，如浏览器、HTTP、HTML、网络服务器、需求 ...

浏览器工作原理？ 百度知道
zhidao.baidu.com › 电脑/网络 › 互联网 - 网页快照
2 个回答 - 2006年1月6日
最佳答案： WWW 的工作基于客户机/服务器计算模型，由Web 浏览器(客户机)和Web服务器(服务器)构成，两者之间采用超文本传送协议（HTTP）进行通信， HTTP协议 ...

浏览器工作原理- 豁达无争的性情，坦荡如水的人生！ - ITeye技术网站
minijack.iteye.com/blog/1081855 - 网页快照
2011年6月12日 – 首先对上篇blog进行一个补充： 以我做的"基于线程池和数据库连接池的web服务器"为例，说说HTTP通信的流程，大体分为三个阶段： A． 连接 ...

浏览器是如何工作的？（工作原理） - IE浏览器中文网站
www.iefans.net/liulanqi-ruhe-gongzuo-yuanli/ - 网页快照
2011年10月4日 – 简介浏览器可以被认为是使用最广泛的软件，本文将介绍浏览器的工作原理，我们将看到，从你在地址栏输入google.com到你看到google主页过程中 ...

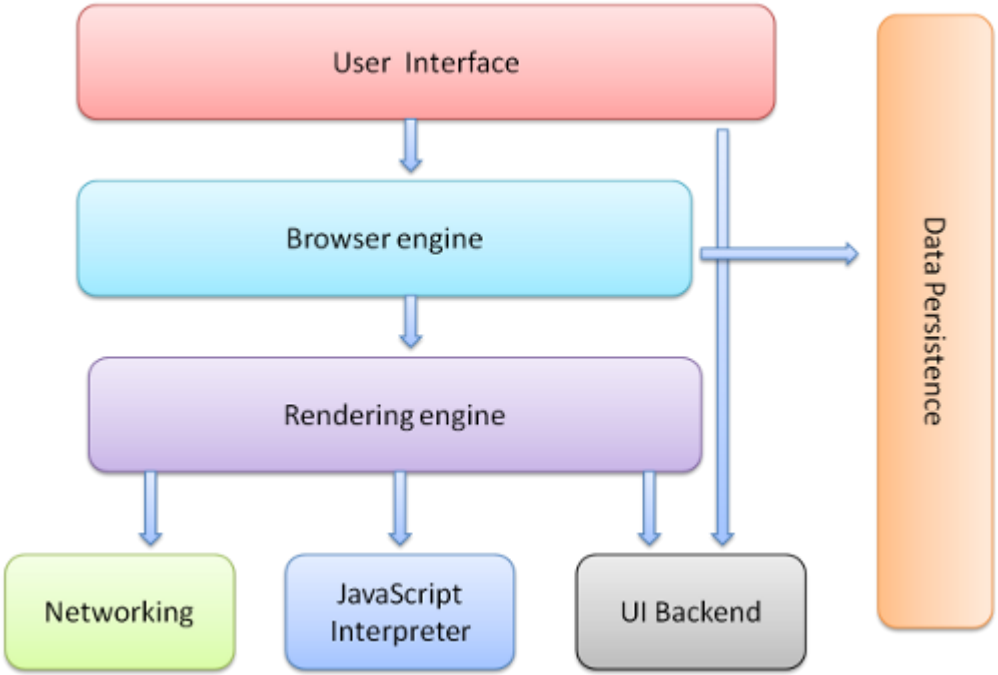
现代浏览器的工作原理【一】 - HavenZhao的专栏- 博客频道- CSDN.NET
blog.csdn.net/beyondhaven/article/details/6755547 - 网页快照

浏览器主要组件

用户界面

浏览器引擎

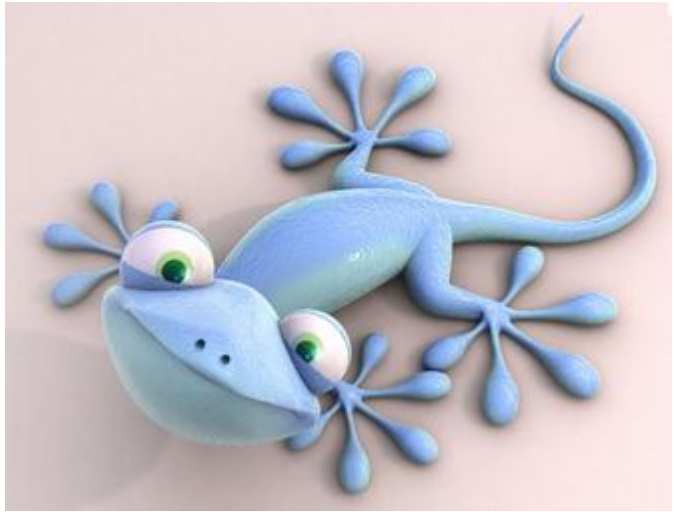
渲染引擎



数据存储

网络　　JS解释器　　UI 后端

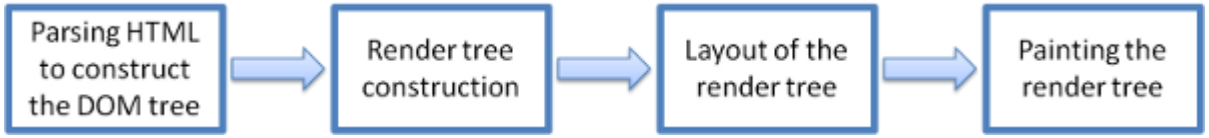
渲染引擎



Gecko

webkit

渲染引擎基本流程



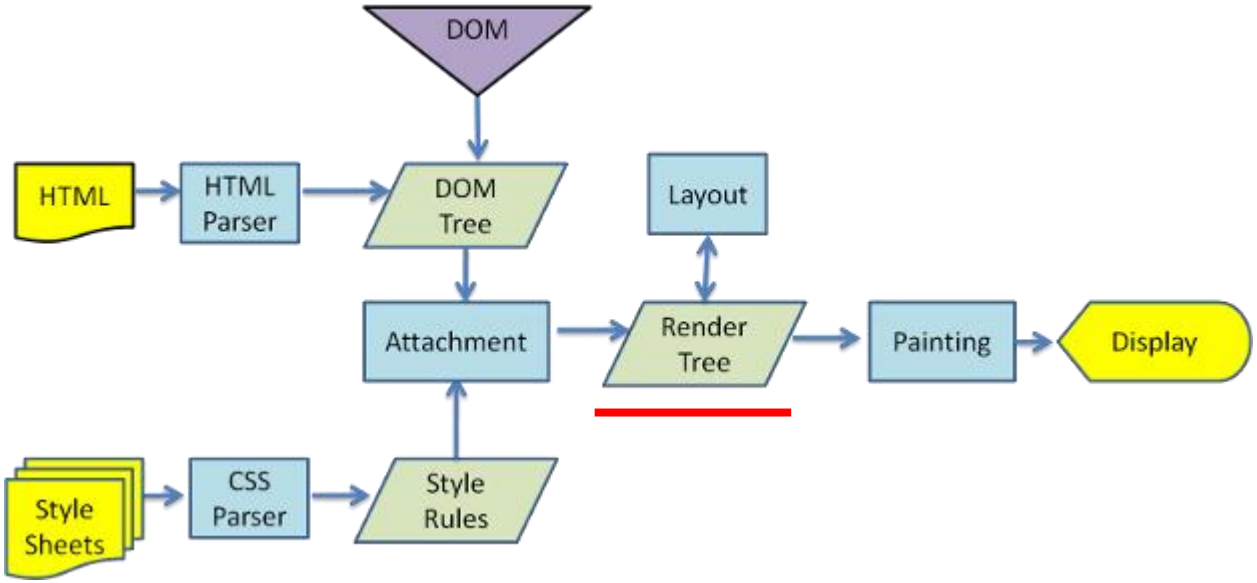
构建dom树

->构建render树

->布局render树

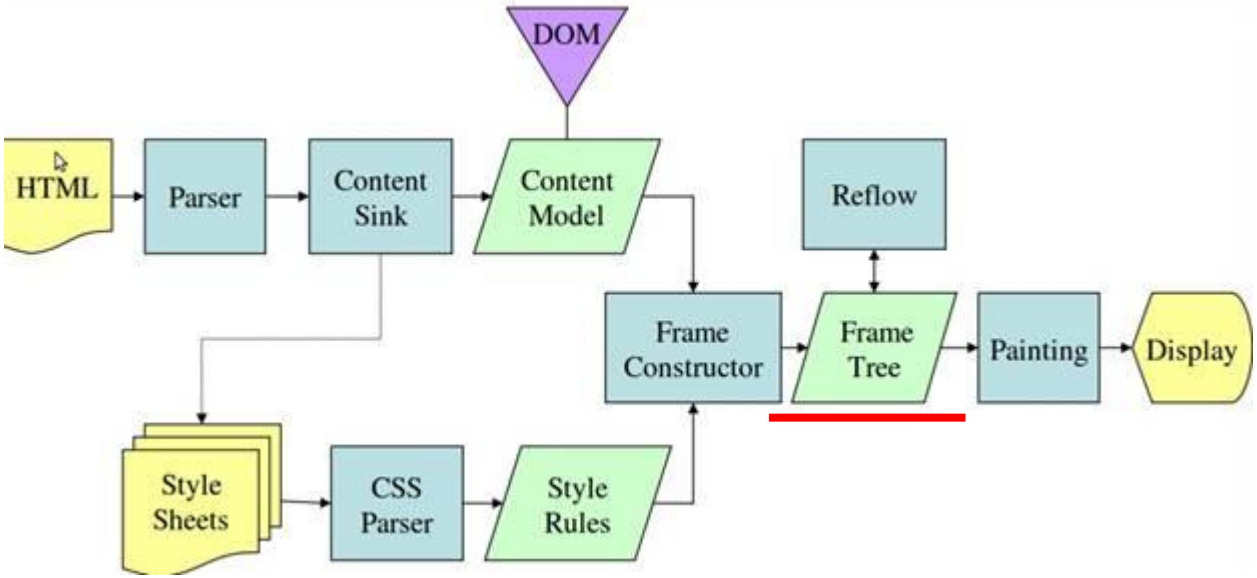
->绘制render树

webkit渲染引擎主流程



- webkit使用 **render树** 这个名词来命名由渲染对象组成的树。
- 元素的定位称为 **布局**。
- 利用dom节点及样式信息去构建render树的过程为 **attachment**

Gecko渲染引擎主流程



- Gecko称可见的格式化元素组成的树为 **frame树**，每个元素都是一个frame。
- 元素的定位称为 **回流**。
- Gecko在html和dom树之间附加了一层，这层称为 **内容接收器**，相当制造dom元素的工厂。

财付通设计中心
Tenpay Innovation Design

解析流程

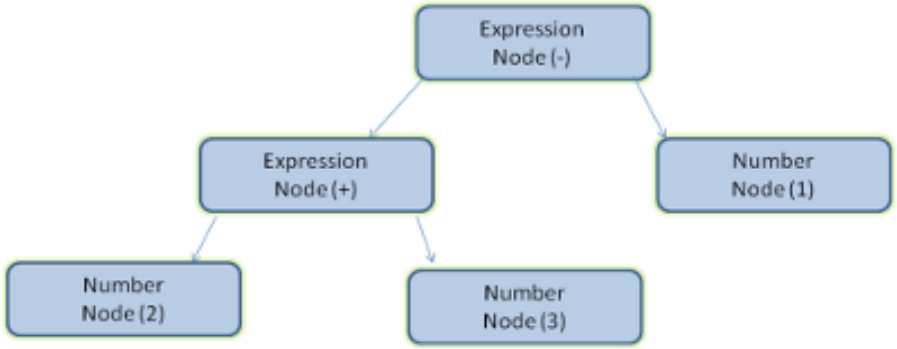
浏览器引擎

解析

解析一个文档：即将其转换为**具有一定意义**的结构——编码可以理解和使用的东西。

解析的结果通常是表达文档结构的节点树，称为**解析树**或**语法树**。

例如，解析“2 + 3 - 1”这个表达式，可能返回这样一棵树。

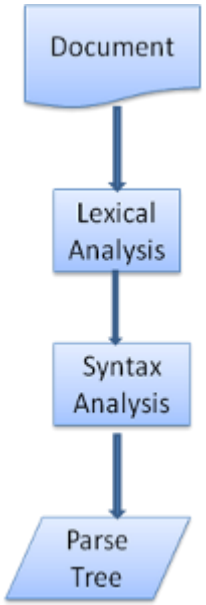


数学表达式树节点

解析

- 解析可以分为两个子过程
  - 语法分析：对语言应用语法规则。
  - 词法分析：将输入分解为符号，符号是语言的词汇表——基本有效单元的集合。

解析器 - 词法分析及语法分析



原文件

词法分析

语法分析

解析树

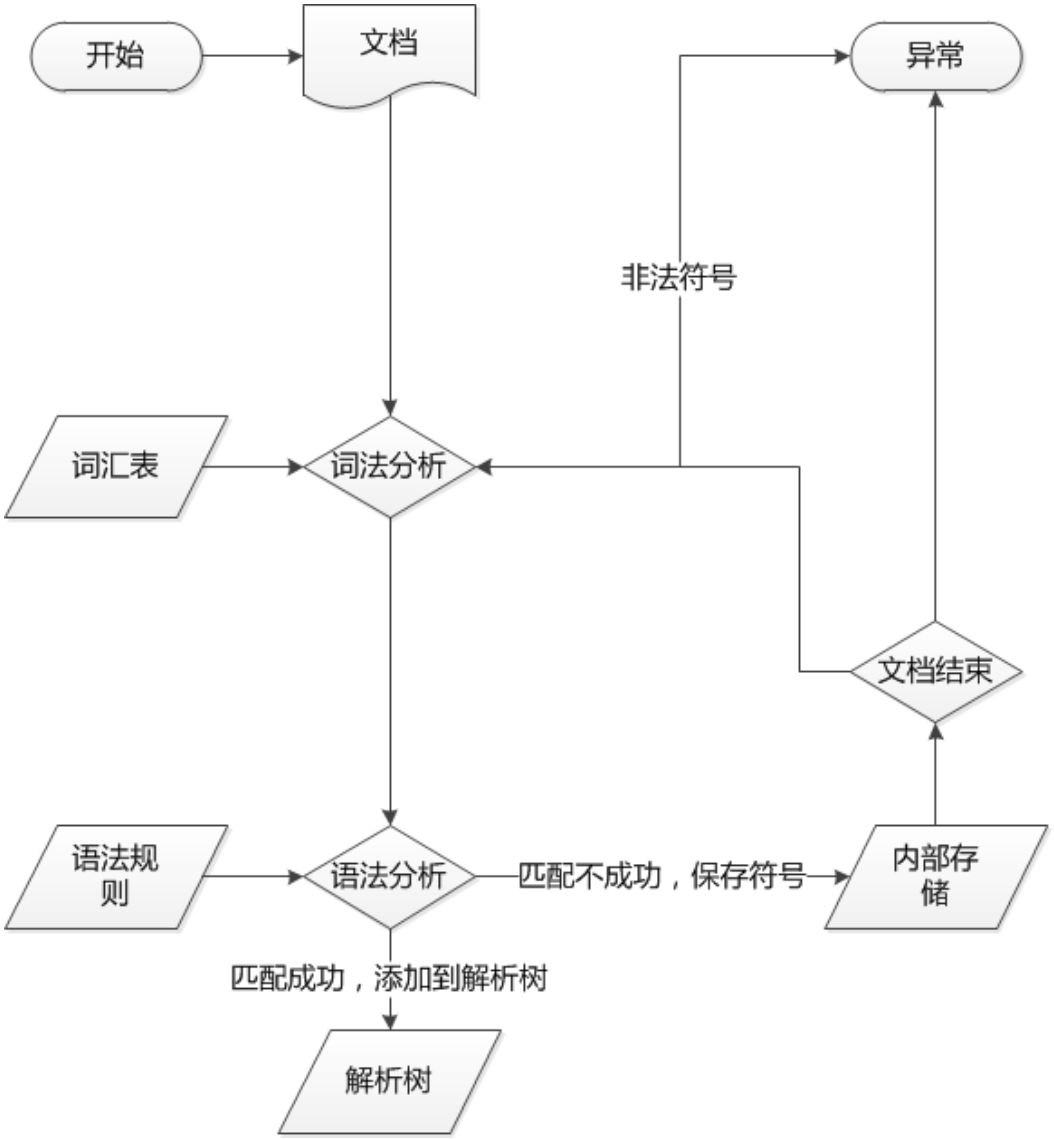
从源文档到解析树

解析器 - 文法

- 上下文无关文法
  - 如果一个语言的文法是上下文无关的，则它可以用正则解析器来解析。对上下文无关文法的一个直观的定义是，该文法可以用BNF来完整的表达。

BNF：http://en.wikipedia.org/wiki/Context-free_grammar

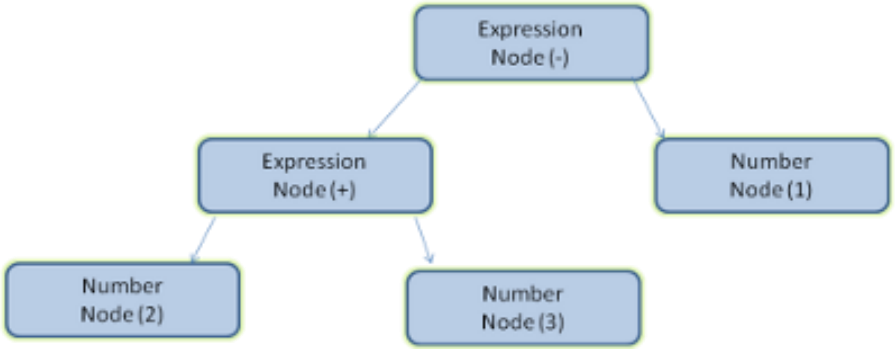
从源文档到解析树


开始
文档
异常
非法符号
词汇表
词法分析
文档结束
语法规则
语法分析
匹配不成功，保存符号
内部存储
匹配成功，添加到解析树
解析树

解析器类型

- 解析器类型分为两种：
  - 自顶向下解析：查看语法的最高层结构并试着匹配其中一个；
  - 自底向上解析：从输入开始，逐步将其转换为语法规则，从底层规则开始直到匹配高层规则。
- Webkit使用自底向上的解析器，Gecko使用自顶向下的解析器

例子：解析“2 + 3 - 1”这个表达式



词汇表：包括整数、加号及减号。

语法：

1. 该语言的语法基本单元包括表达式、term及操作符
2. 该语言可以包括多个表达式
3. 一个表达式定义为两个term通过一个操作符连接
4. 操作符可以是加号或减号
5. term可以是一个整数或一个表达式

例子：词汇表及语法的定义

词汇表：包括整数、加号及减号。

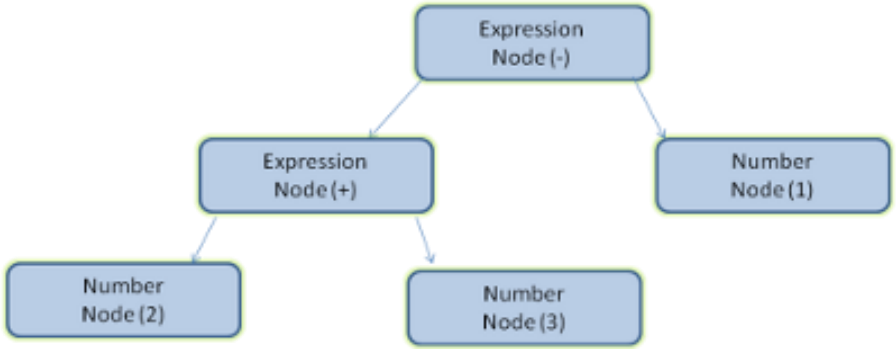
INTEGER：0 | [1 - 9] [0 - 9] *
PLUS： +
MINUS： -

语法：

1. 该语言的语法基本单元包括表达式、term及操作符
2. 该语言可以包括多个表达式
3. 一个表达式定义为两个term通过一个操作符连接
4. 操作符可以是加号或减号
5. term可以是一个整数或一个表达式

expression := term operation term
operation := PLUS | MINUS
term := INTEGER | expression

例子：解析过程



1. 第一个匹配规则的子字符串是“2”，根据规则5，它是一个term
2. 第二个匹配的是“2 + 3”，它符合第2条规则——一个操作符连接两个term，下一次匹配发生在输入的结束处。
3. “2 + 3 - 1”是一个表达式，因为我们已经知道“2 + 3”是一个term，所以我们有了一个term紧跟着一个操作符及另一个term。
4. “2 + +”将不会匹配任何规则，因此是一个无效输入。

HTML解析

非上下文无关文法

HTML 解析

- **HTML文法定义——HTML DTD**
  - DTD（Document Type Definition 文档类型定义）
  - 这一格式是用于定义SGML家族的语言，包括了对所有允许元素及它们的属性和层次关系的定义。
  - DTD定义了HTML的解析语法

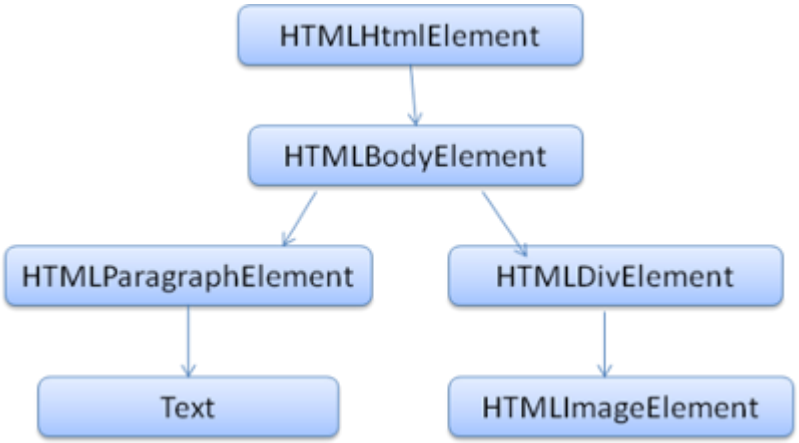
HTML 解析树——DOM树

- HTML解析器输出的树，也就是解析树，是由DOM元素及属性节点组成的。
- DOM是文档对象模型的缩写，它是html文档的对象表示，作为html元素的外部接口供js等调用。

http://www.w3.org/DOM/DOMTR

例子：DOM树

```
<html>
<body>
<p>Hello DOM</p>
<div><img src="example.png" /></div>
</body>
</html>
```

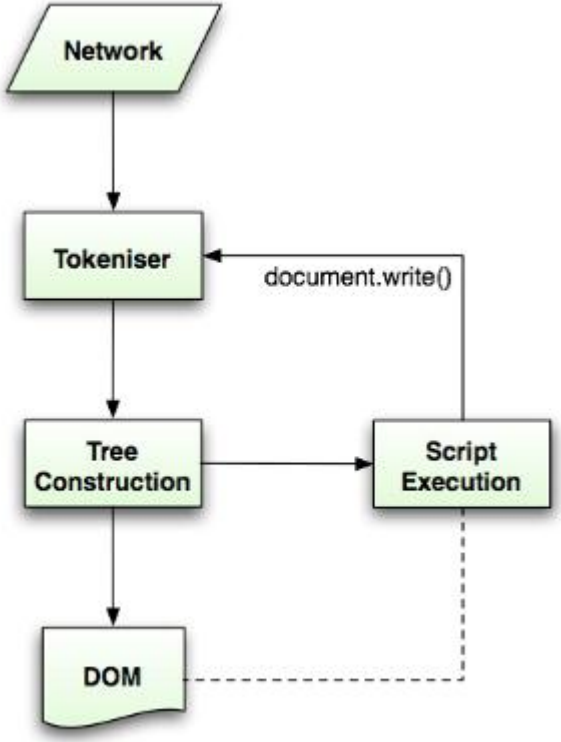


这里所谓的树包含了DOM节点是说树是由 **实现了DOM接口的元素** 构建而成的，浏览器使用已被浏览器内部使用的其他属性的具体实现。

解析算法

- hmtl不能被一般的自顶向下或自底向上的解析器所解析。
  - 原因是：
    - 1. 这门语言本身的宽容特性
    - 2. 浏览器对一些常见的非法html有容错机制
    - 3. 解析过程是往复的，通常源码不会在解析过程中发生改变，但在html中，脚本标签包含的“document.write”可能添加标签，这说明在解析过程中实际上修改了输入
- 不能使用正则解析技术，浏览器为html定制了专属的解析器。
- Html5规范中描述了这个解析算法，算法包括两个阶段——**符号化及构建树**。

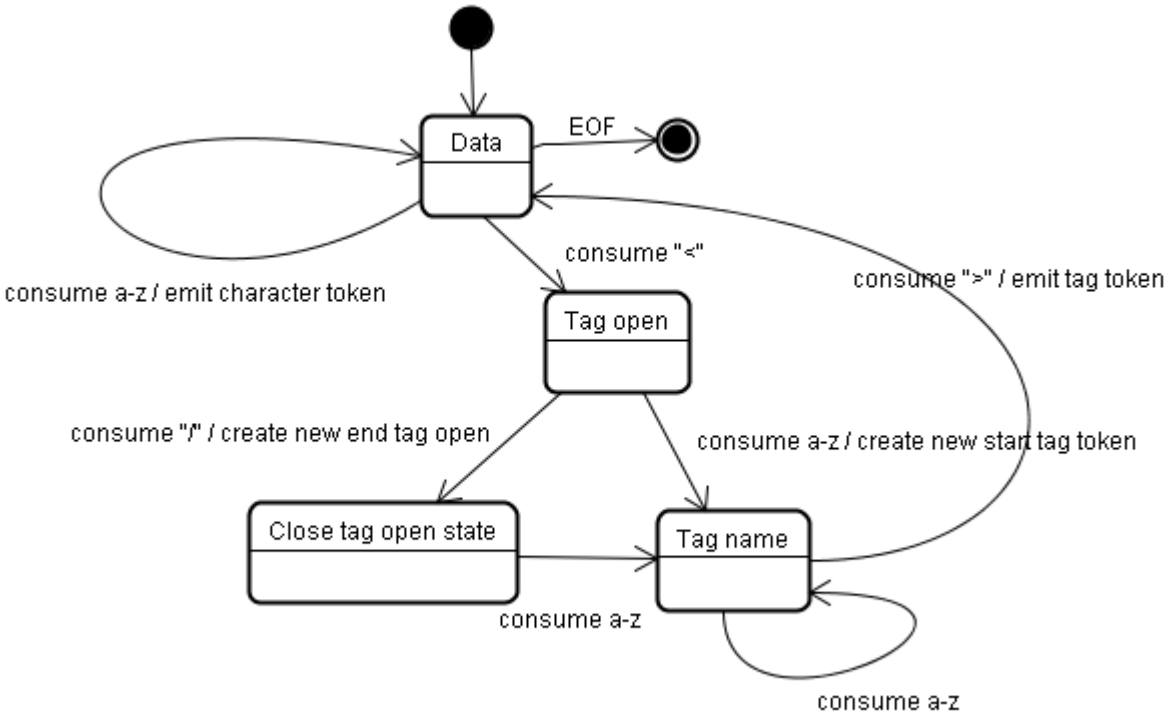
符号化



- 符号化是词法分析的过程，将输入解析为符号，html的符号包括开始标签、结束标签、属性名及属性值。
- 符号识别器识别出符号后，将其传递给树构建器，并读取下一个字符，以识别下一个符号，这样直到处理完所有输入。

符号化：词法分析

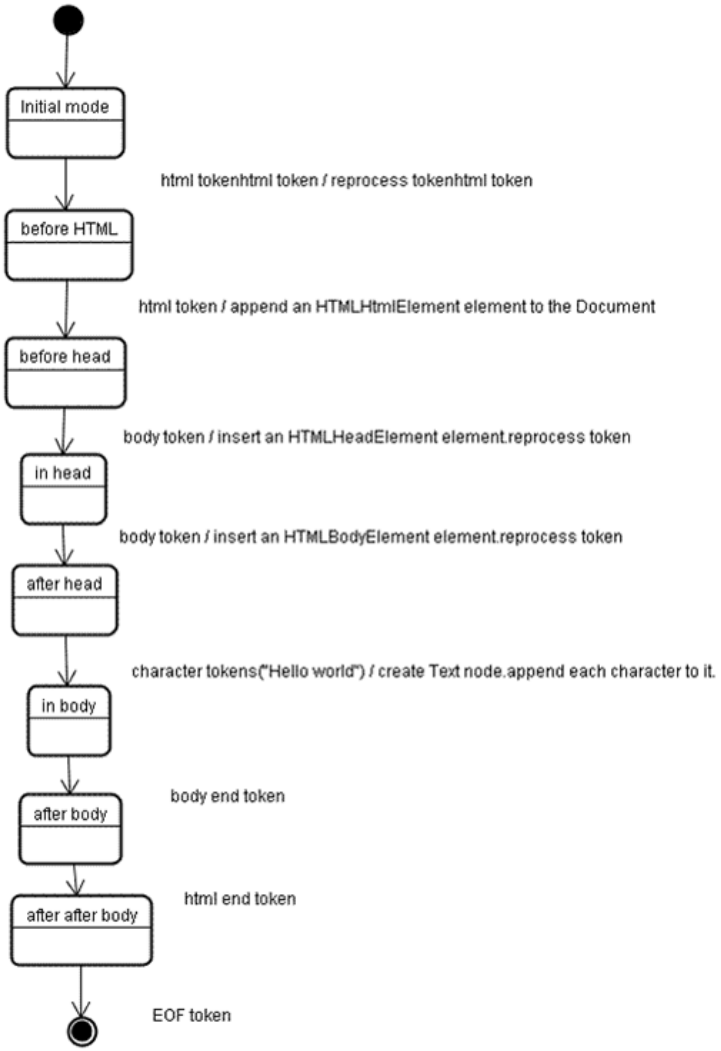
```
<html>
<body>
Hello world
</body>
</html>
```



算法输出html符号，该算法用状态机表示。

构建树

```
<html>
<body>
Hello world
</body>
</html>
```



构建树这一阶段的输入是符号识别阶段生成的符号序列。

财付通设计中心
Tenpay Innovation Design

解析结束时的处理

- 在这个阶段，浏览器将文档标记为可交互的，并开始解析处于延时模式中的脚本——这些脚本在文档解析后执行。
- 文档状态将被设置为完成，同时触发一个load事件。

Html5规范中有符号化及构建树的完整算法
(http://www.w3.org/TR/html5/syntax.html#html-parser)。

浏览器容错

- 浏览器都具有错误处理的能力，但是，另人惊讶的是，这并不是html最新规范的内容，就像书签及前进后退按钮一样，它只是浏览器长期发展的结果。
- 解析器将符号化的输入解析为文档并创建文档，但不幸的是，我们必须处理很多没有很好格式化的html文档，至少要小心下面几种错误情况。
  - 标签未关闭
  - 标签嵌套错误
  - 标签错误
  - 遗漏标签
  - 太深的标签继承，最多只允许20个相同类型的标签嵌套。

CSS解析

上下文无关文法

CSS解析 - 词法

comment ///*[^*]*/*+([^/*][^*]*/*+)*//

num [0-9]+|[0-9]*”.”[0-9]+

nonascii [/200-/377]

nmstart [_a-z]|{nonascii}|{escape}

nmchar [_a-z0-9-]|{nonascii}|{escape}

name {nmchar}+

ident {nmstart}{nmchar}*

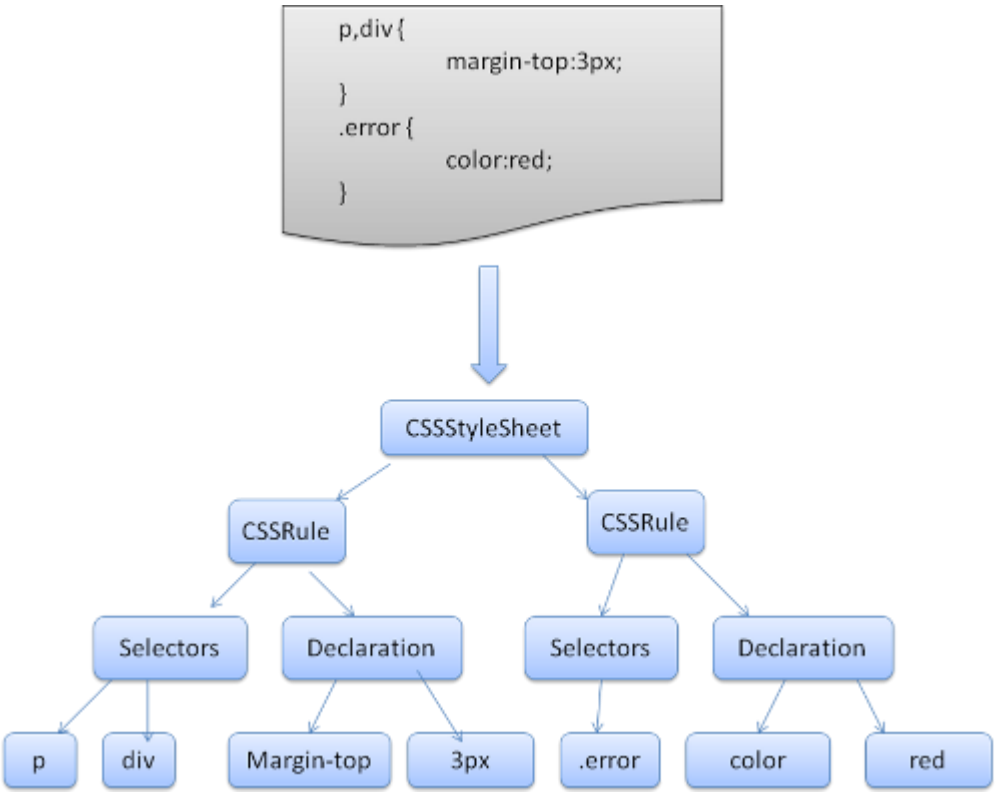
CSS解析 - 语法

```
ruleset
: selector [ ',' S* selector ]*
'{' S* declaration [ ';' S* declaration ]* '}' S*
;
selector
: simple_selector [ combinator selector | S+ [ combinator selector ] ]
;
simple_selector
: element_name [ HASH | class | attrib | pseudo ]*| [ HASH | class | attrib | pseudo ]+
;
class
: '.' IDENT
;
element_name
: IDENT | '*'
;
attrib
: '[' S* IDENT S* [ [ '=' | INCLUDES | DASHMATCH ] S*[ IDENT | STRING ] S* ] ']'
;
pseudo
: ':' [ IDENT | FUNCTION S* [IDENT S*] ')' ]
;
```

“ident”是识别器的缩写，相当于一个class名，“name”是一个元素id（用“#”引用）。

CSS解析器

- Webkit使用自底向上的解析器，Gecko使用自顶向下的解析器
- 它们都是将每个css文件解析为样式表对象，每个对象包含css规则，css规则对象包含选择器和声明对象，以及其他一些符合css语法的对象。



脚本解析

javascript

脚本

- web的模式是同步的，开发者希望解析到一个script标签时立即解析执行脚本，并阻塞文档的解析直到脚本执行完。
- 如果脚本是外引的，则网络必须先请求到这个资源——这个过程也是同步的，会阻塞文档的解析直到资源被请求到。这个模式保持了很多年，并且在html4及html5中都特别指定了。
- 开发者可以将脚本标识为defer，以使其不阻塞文档解析，并在文档解析结束后执行。Html5增加了标记脚本为异步的选项，以使脚本的解析执行使用另一个线程。

预解析

- Webkit和Firefox都做了这个优化，当执行脚本时，另一个线程解析剩下的文档，并加载后面需要通过网络加载的资源。
- 这种方式可以使资源并行加载从而使整体速度更快。
- 需要注意的是，预解析并不改变Dom树，它将这个工作留给主解析过程，自己只解析外部资源的引用，比如外部脚本、样式表及图片。

样式表

- 样式表采用另一种不同的模式。
- 理论上，既然样式表不改变Dom树，也就没有必要停下文档的解析等待它们，然而，存在一个问题，脚本可能在文档的解析过程中请求样式信息，如果样式还没有加载和解析，脚本将得到错误的值，显然这将会导致很多问题，这看起来是个边缘情况，但确实很常见。
- Firefox在存在样式表还在加载和解析时阻塞所有的脚本，而chrome只在当脚本试图访问某些可能被未加载的样式表所影响的特定的样式属性时才阻塞这些脚本。

渲染树的构造

渲染引擎

渲染树

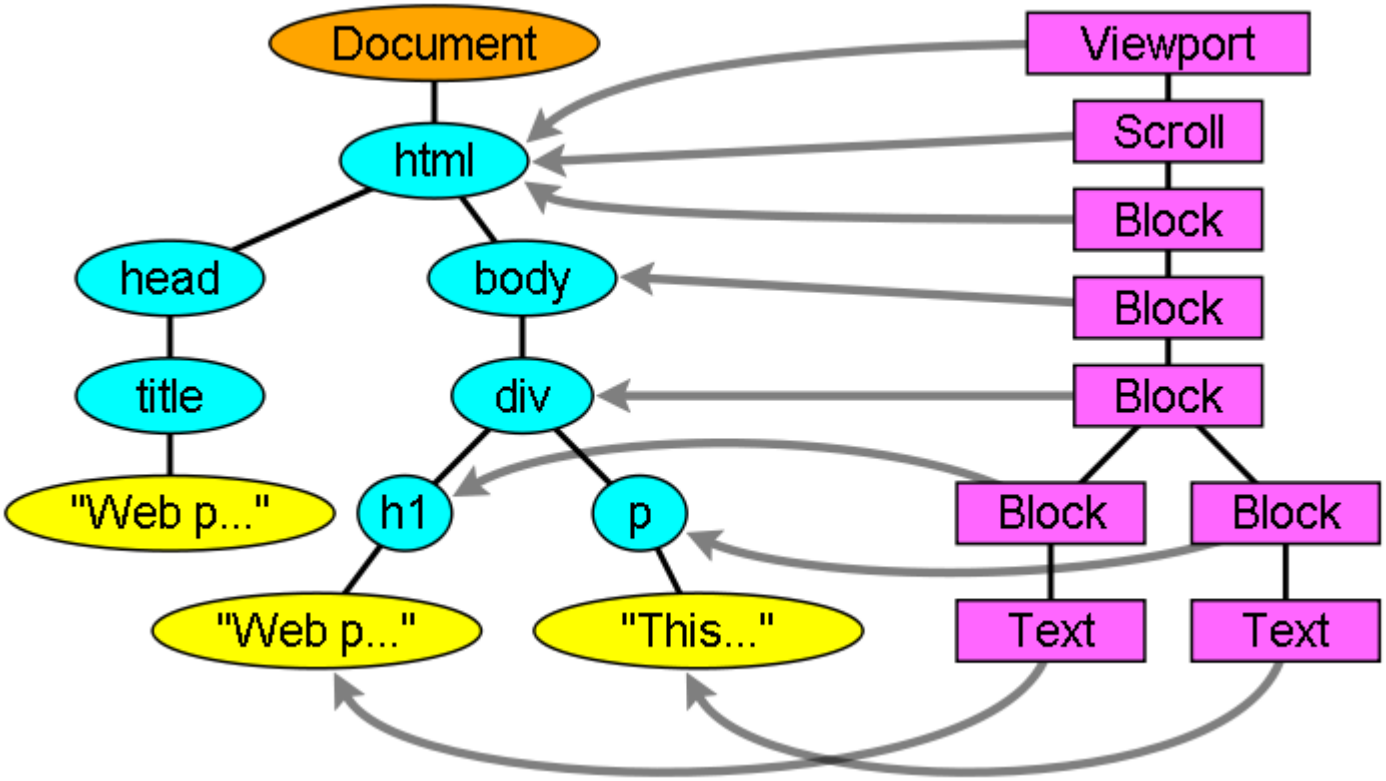
- 渲染树由**元素显示序列中的可见元素组成**，它是文档的可视化表示，构建这棵树是为了以正确的顺序绘制文档内容。
- Firefox将渲染树中的元素称为frames，webkit则用render或渲染对象来描述这些元素。
- 一个渲染对象知道怎么布局及绘制自己及它的children。每个渲染对象用一个和该节点的css盒模型相对应的**矩形区域**来表示，正如css2所描述的那样，它包含诸如宽、高和位置之类的几何信息。盒模型的类型受该节点相关的display样式属性的影响。元素的类型也需要考虑，例如，表单控件和表格带有特殊的框架。

渲染树和DOM树的关系

- 渲染对象和DOM元素相对应，但这种对应关系**不是一对一**的，**不可见的Dom元素不会被插入渲染树**，例如head元素。另外，**display属性为none的元素也不会在渲染树中出现**（visibility属性为hidden的元素将出现在渲染树中）。
- 还有一些Dom元素对应几个可见对象，它们一般是一些具有复杂结构的元素，无法用一个矩形来描述。例如，select元素有三个渲染对象——一个显示区域、一个下拉列表及一个按钮。
- 当文本因为宽度不够而折行时，新行将作为额外的渲染元素被添加。
- 根据css规范，一个行内元素只能仅包含行内元素或仅包含块状元素，在存在混合内容时，将会创建匿名的块状渲染对象包裹住行内元素。

渲染树和DOM树的关系

- 一些渲染对象和所对应的Dom节点不在树上相同的位置，例如，浮动和绝对定位的元素在文本流之外，在两棵树上的位置不同，渲染树上标识出真实的结构，并用一个占位结构标识出它们原来的位置。



样式计算

- 创建渲染树需要计算出每个渲染对象的可视属性，这可以通过计算每个元素的样式属性得到。
- 样式包括各种来源的样式表，行内样式元素及html中的可视化属性（例如bgcolor），可视化属性转化为css样式属性。
- 样式表来源于浏览器默认样式表，及页面作者和用户提供的样式表，有些样式是浏览器用户提供的（浏览器允许用户定义喜欢的样式，例如，在Firefox中，可以通过在Firefox Profile目录下放置样式表实现）。

计算样式的一些困难

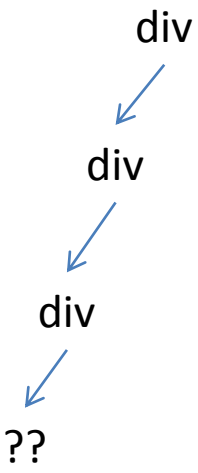
计算样式的一些困难

- 样式数据是非常大的结构，保存大量的样式属性会带来内存问题
- 元素查找匹配的规则的优化
- 应用规则涉及非常复杂的级联

例子：复杂的选择符

div div div div {…}

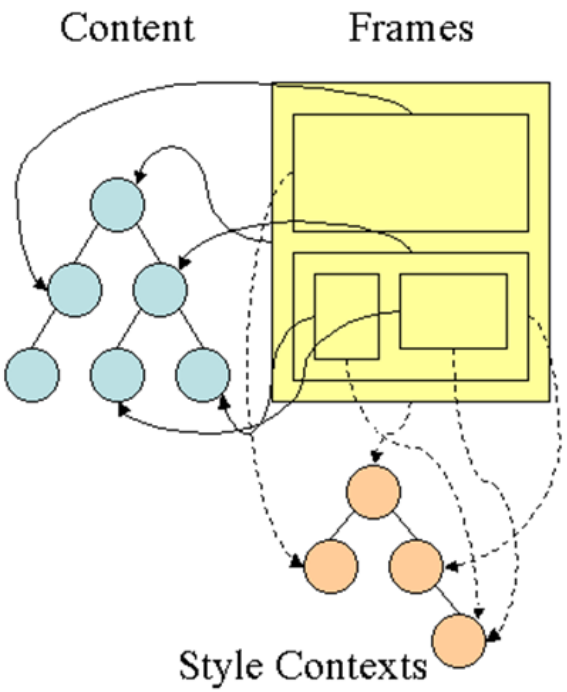
```
<div>
   <div>
      <div>cont</div>
   </div>
</div>
```



Gecko - 规则树、上下文树

Gecko - 规则树、上下文树

- Gecko用两个树用来简化样式计算 - 规则树和样式上下文树



webkit也有样式对象，但它们并没有存储在类似样式上下文树这样的树中，只是由Dom节点指向其相关的样式。

例子：使用规则树计算样式上下文

```
<html>
<body>
<div class=”err” id=”div1”>
<p>this is a
<span class=”big”> big error </span>
this is also a
<span class=”big”> very big error</span>
error
</p>
</div>
<div class=”err” id=”div2”>another error</div>
</body>
</html>
```

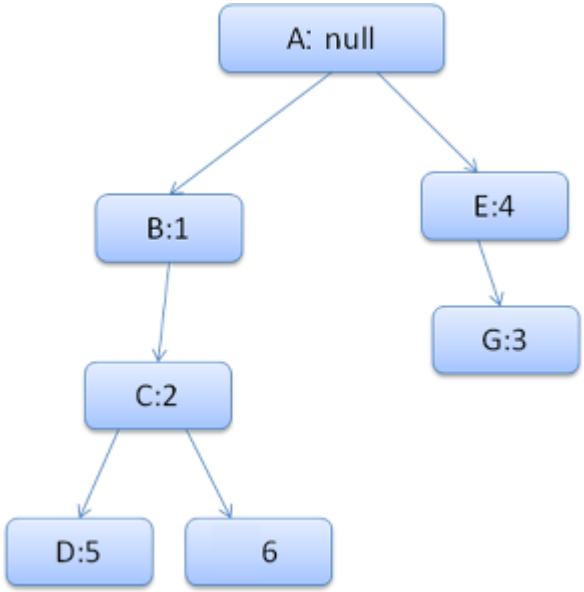
1. div {margin:5px;color:black}
2. .err {color:red}
3. .big {margin-top:3px}
4. div span {margin-bottom:4px}
5. #div1 {color:blue}
6. #div2 {color:green}

简化下问题，我们只填充两个结构——color和margin，color结构只包含一个成员 - 颜色，margin结构包含四边。

例子：规则树（指向规则）

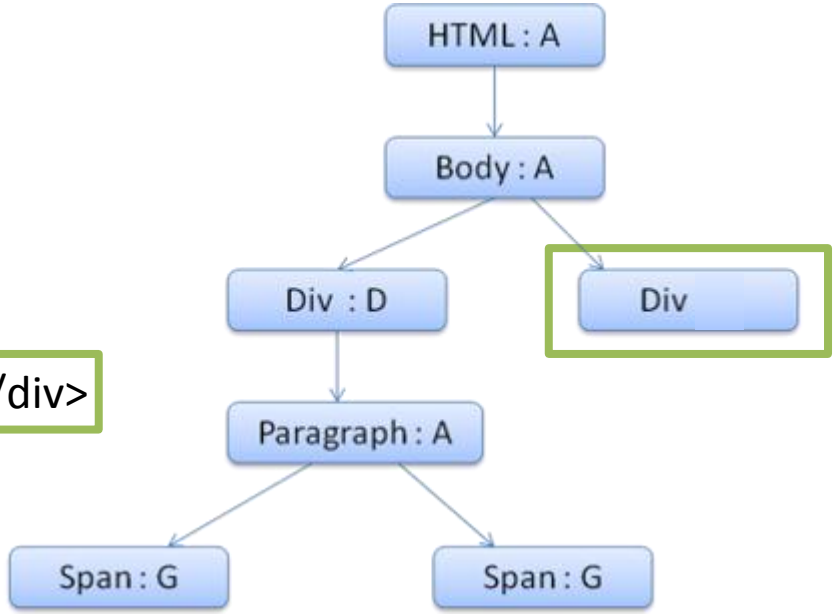
```
<html>
<body>
<div class="err" id="div1">
<p>this is a
<span class="big"> big error </span>
this is also a
<span class="big"> very big error</span>
error
</p>
</div>
<div class="err" id="div2">another error</div>
</body>
</html>
```

1. div {margin:5px;color:black}
2. .err {color:red}
3. .big {margin-top:3px}
4. div span {margin-bottom:4px}
5. #div1 {color:blue}
6. #div2 {color:green}



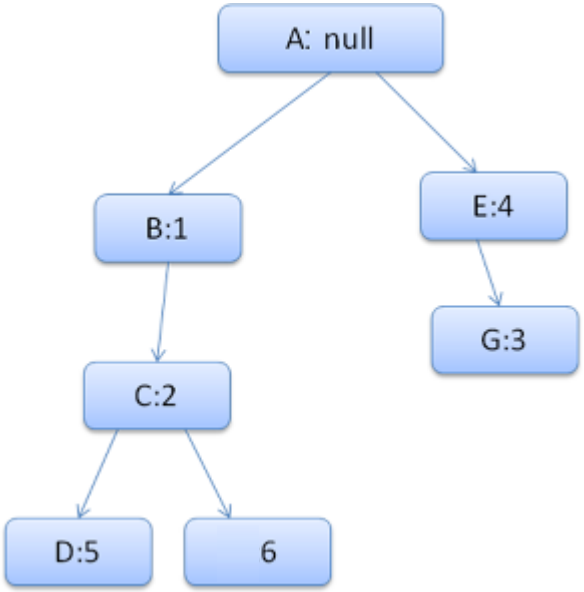
例子：上下文树（指向规则节点）

```
<html>
<body>
<div class=”err” id=”div1”>
<p>this is a
<span class=”big”> big error </span>
this is also a
<span class=”big”> very big error</span>
error
</p>
</div>
<div class=”err” id=”div2”>another error</div>
</body>
</html>
```



例子：

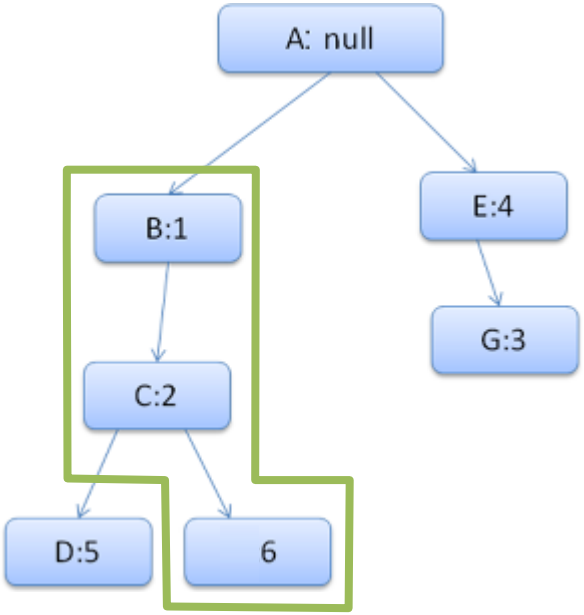
another error



1. div {margin:5px;color:black}
2. .err {color:red}
3. .big {margin-top:3px}
4. div span {margin-bottom:4px}
5. #div1 {color:blue}
6. #div2 {color:green}

例子：

another error



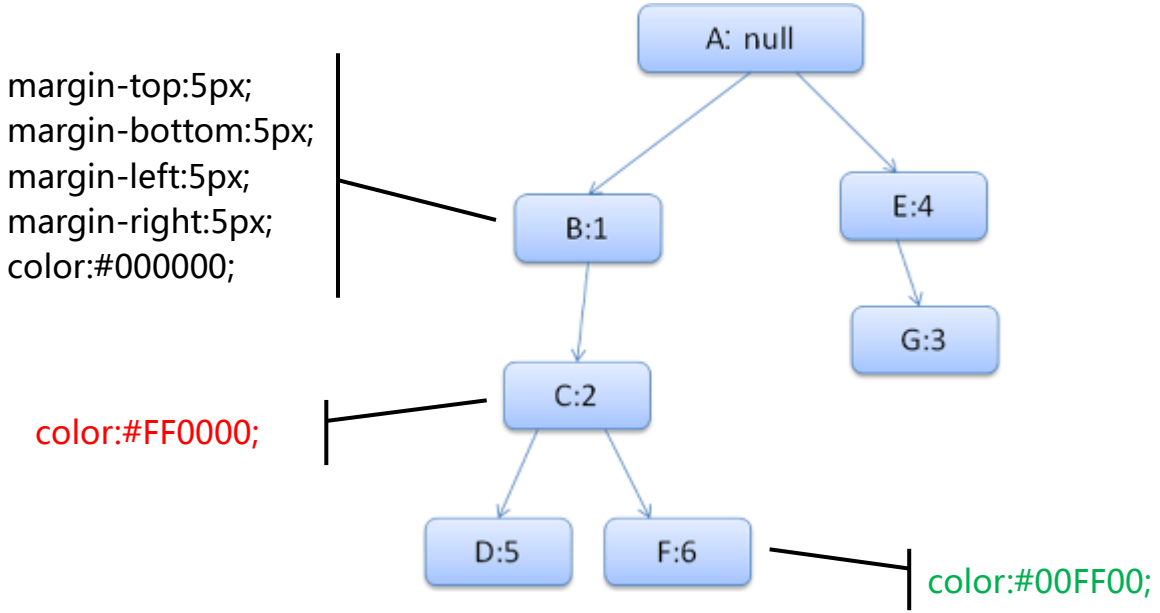
1. 规则树路径：1、2、6
2. 计算规则，缓存结果：
   a. margin-top:5px;
   b. margin-bottom:5px;
   c. margin-left:5px;
   d. margin-right:5px;
   e. color:#00FF00;
3. 创建节点，关联上下文树

结构化

- 样式上下文按结构划分，一个结构中的所有特性不是继承的就是非继承的。
  - 对继承的特性，除非元素自身有定义，否则就从它的parent继承。
  - 非继承的特性（称为reset特性）如果没有定义，则使用默认的值。

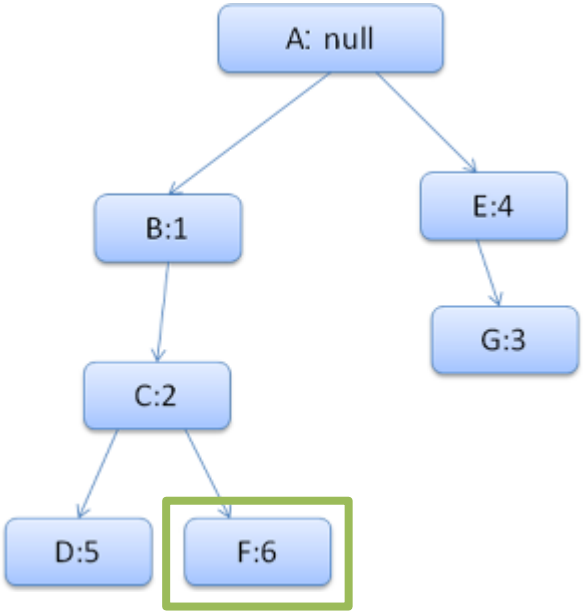
例子：

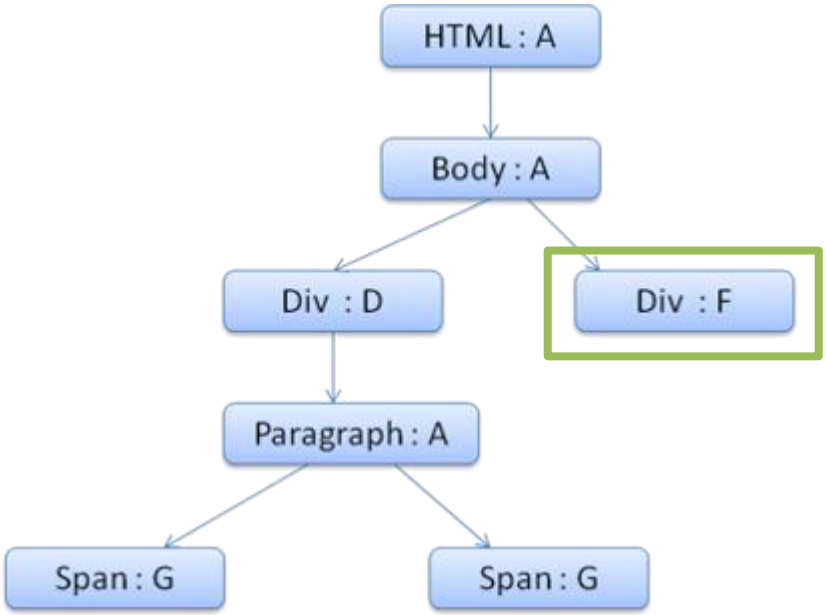
another error



例子：

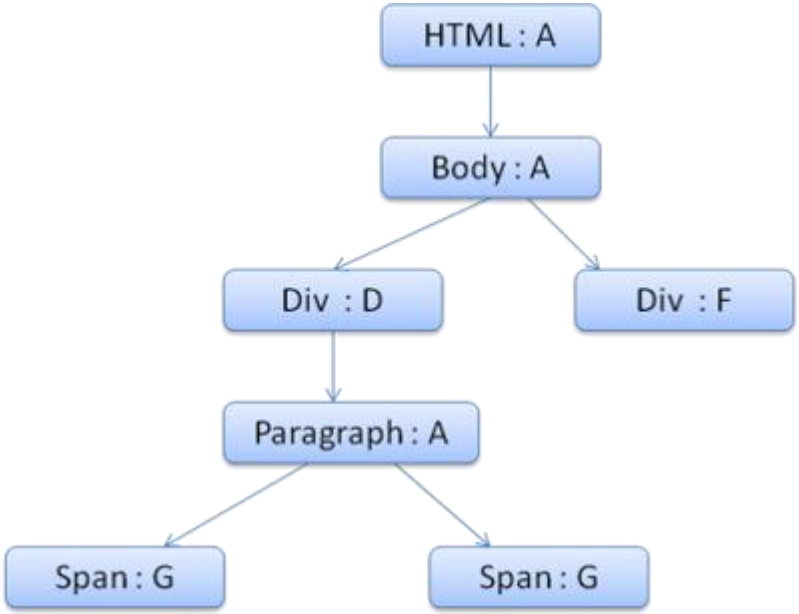
another error





例子：

another error



margin-top:5px;
margin-bottom:5px;
margin-left:5px;
margin-right:5px;
color:#00FF00;

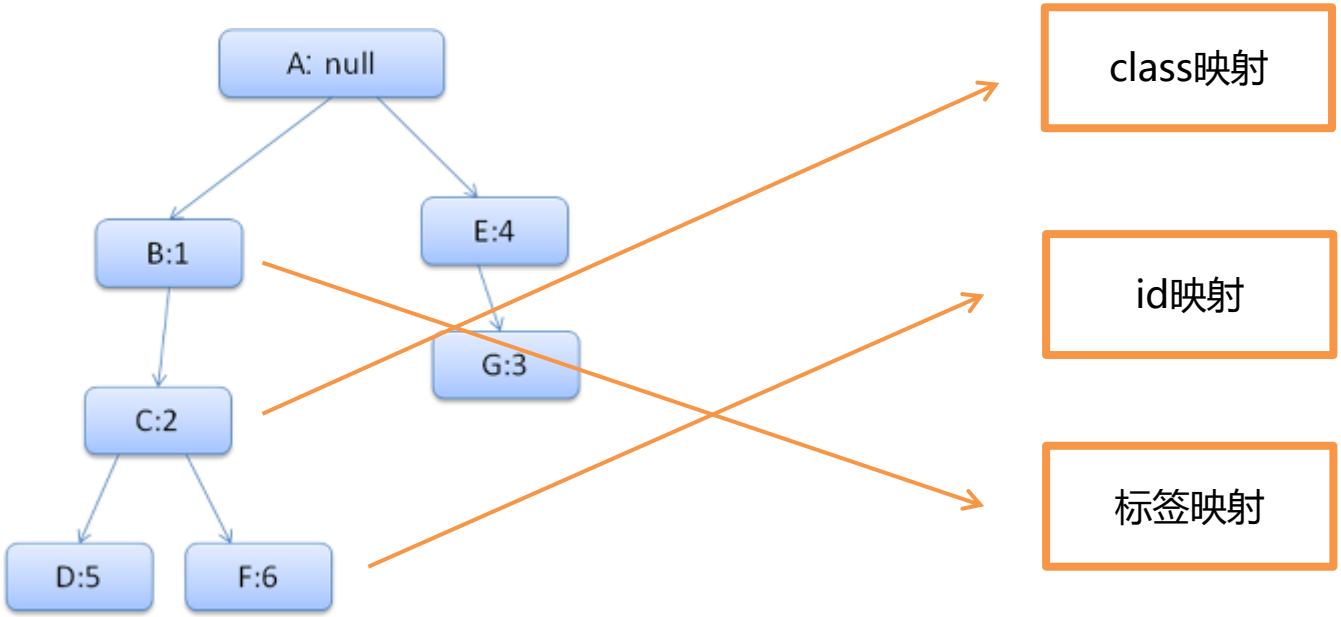
样式规则的来源

- 外部样式或标签内style属性的CSS规则
- 行内样式属性
- html可视化属性（映射为相应的样式规则）

通过映射提高匹配速度

- 样式规则可能很狡猾，为了解决这个问题，可以先对规则进行处理，以使其更容易被访问。
- 解析完样式表之后，规则会根据选择符添加一些hash映射，映射可以是根据id、class、标签名或是任何不属于这些分类的综合映射。
  - 如果选择符为id，规则将被添加到id映射，
  - 如果是class，则被添加到class映射，等等。
- 这个处理是匹配规则更容易，不需要查看每个声明，我们能从映射中找到一个元素的相关规则，这个优化使在进行规则匹配时减少了95 + %的工作量。

例子：映射



1. div {margin:5px;color:black}
2. .err {color:red}
3. .big {margin-top:3px}
4. div span {margin-bottom:4px}
5. #div1 {color:blue}
6. #div2 {color:green}

例子：映射

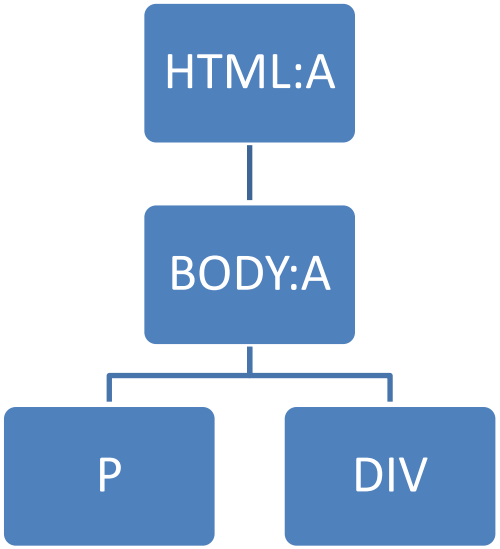
```
<p class=”error”>an error occurred </p>
<div id=”messagediv”>this is a message</div>
```

```
p.error {color:red}
#messagediv {height:50px}
div {margin:5px}
```

例子：映射

an error occurred

this is a message

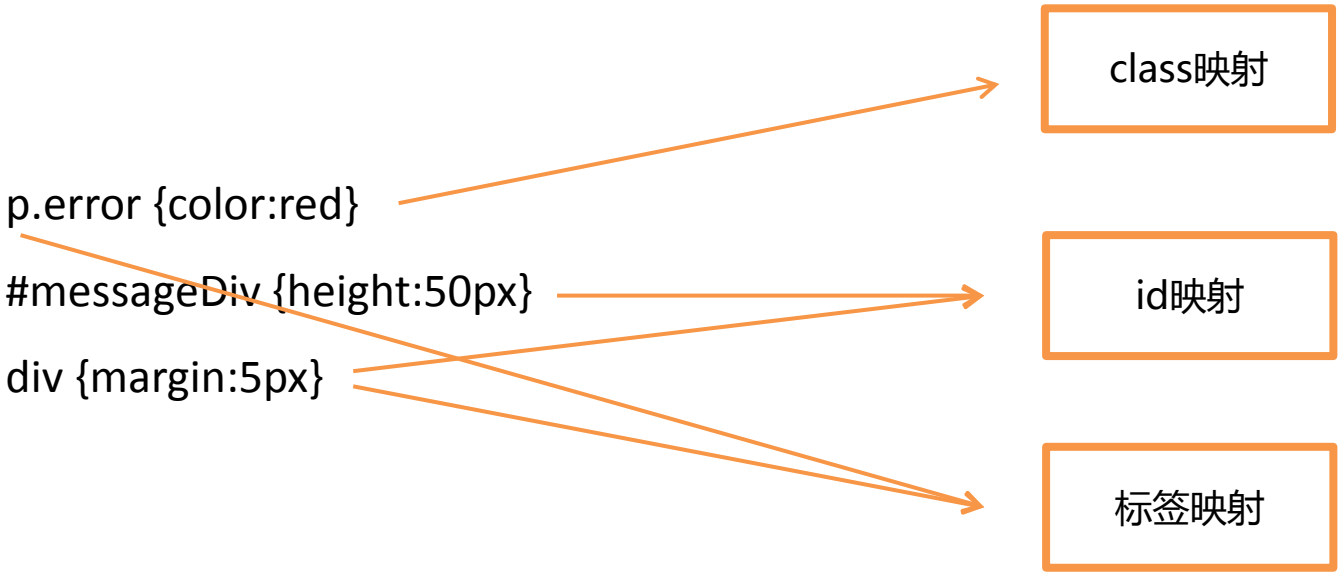


p.error {color:red}

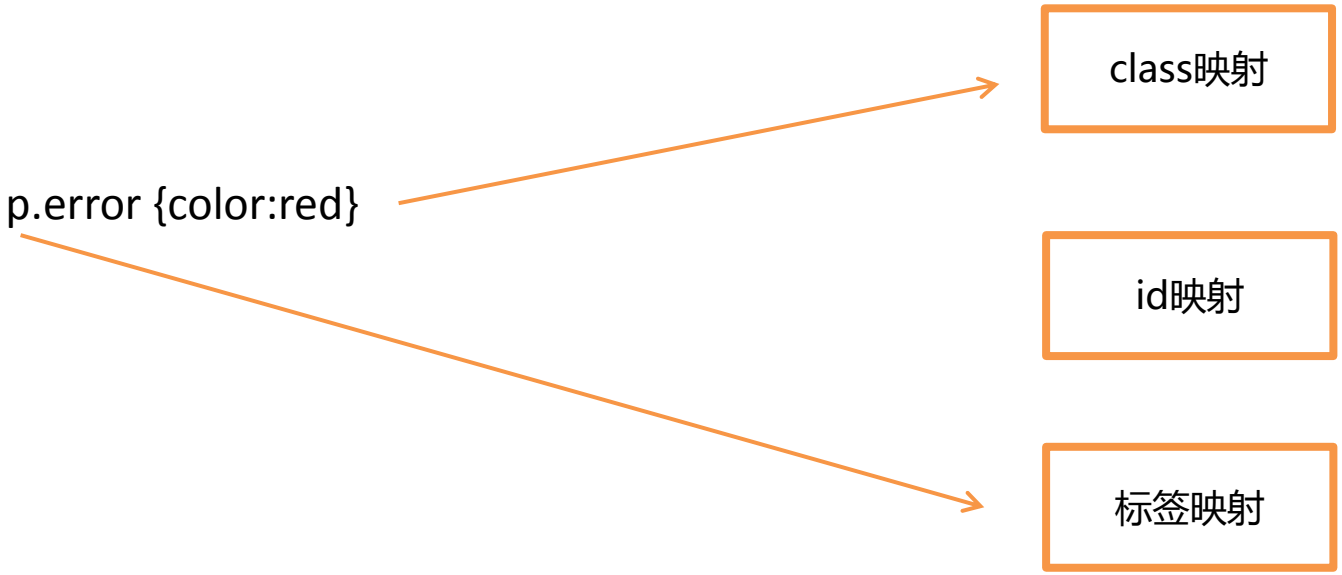
#messagediv {height:50px}

div {margin:5px}

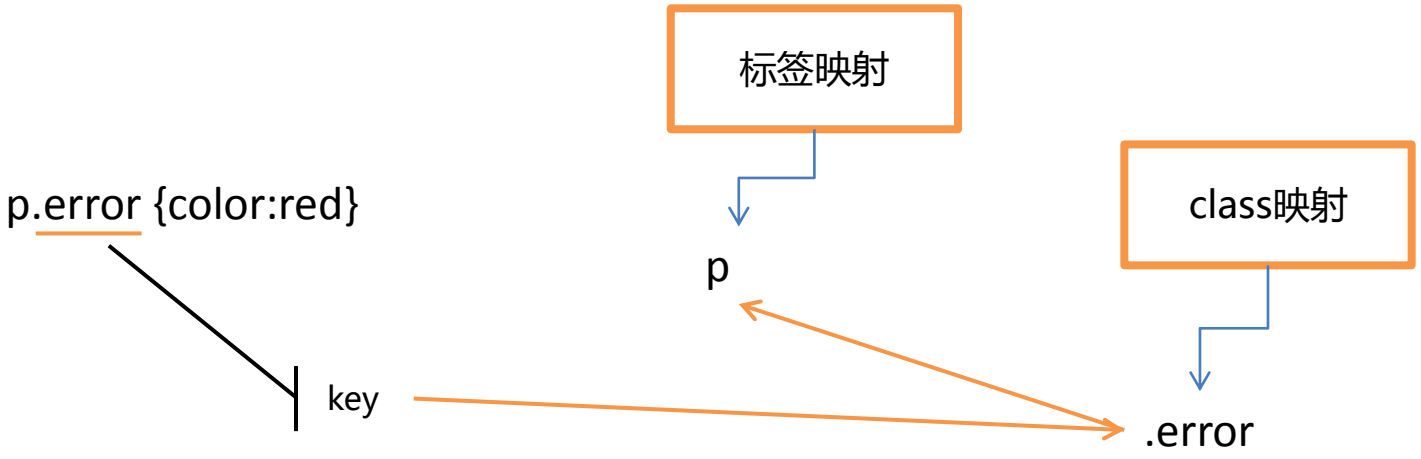
例子：映射


p.error {color:red}
#messageDiv {height:50px}
div {margin:5px}
class映射
id映射
标签映射

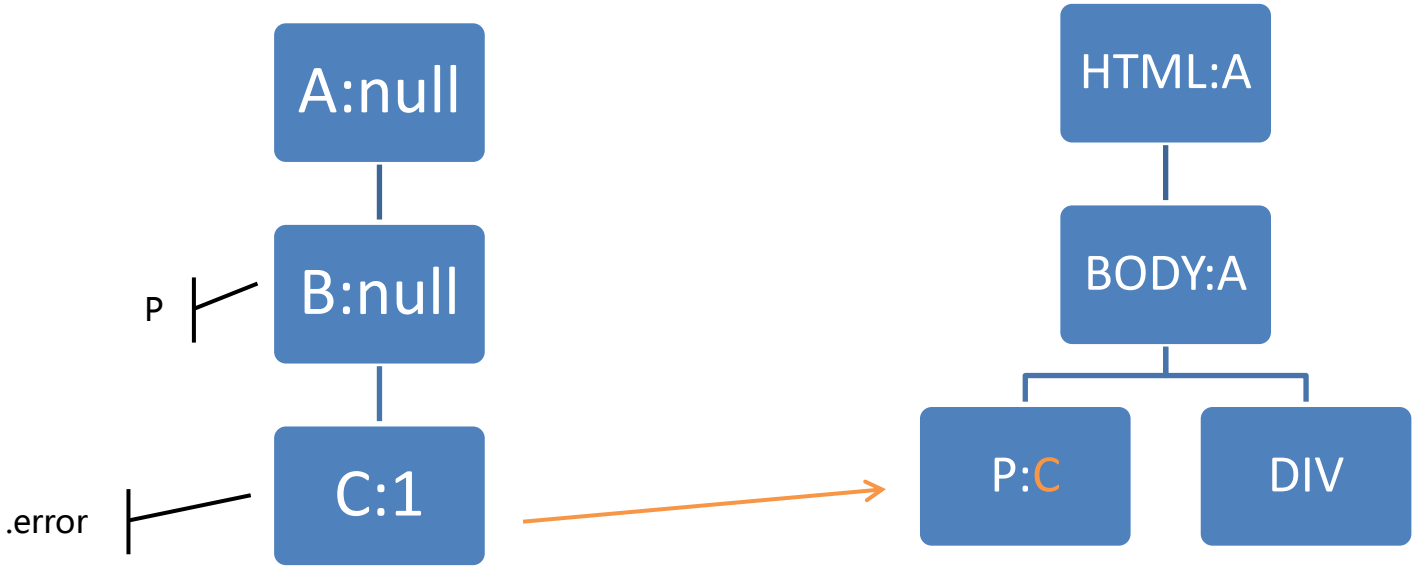
例子：映射



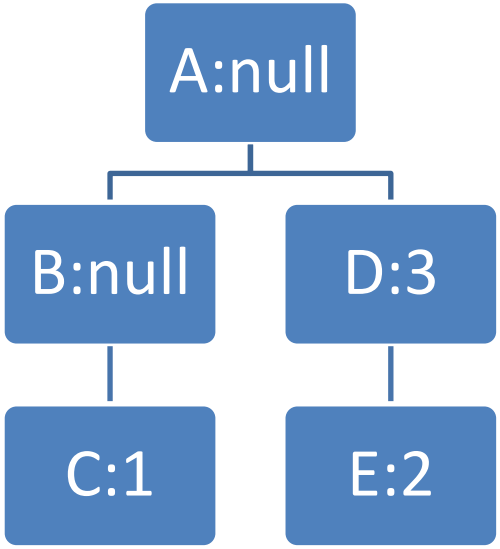
例子：映射

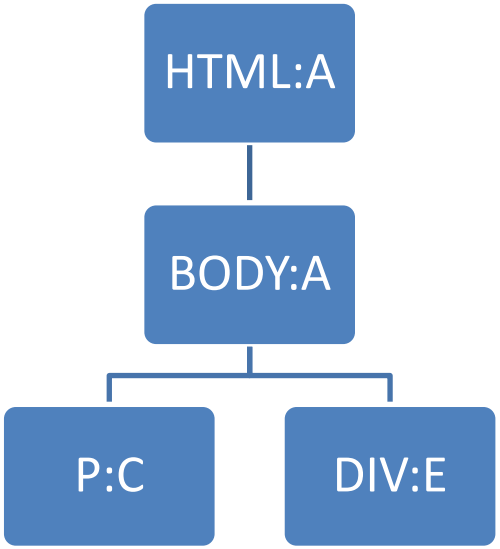


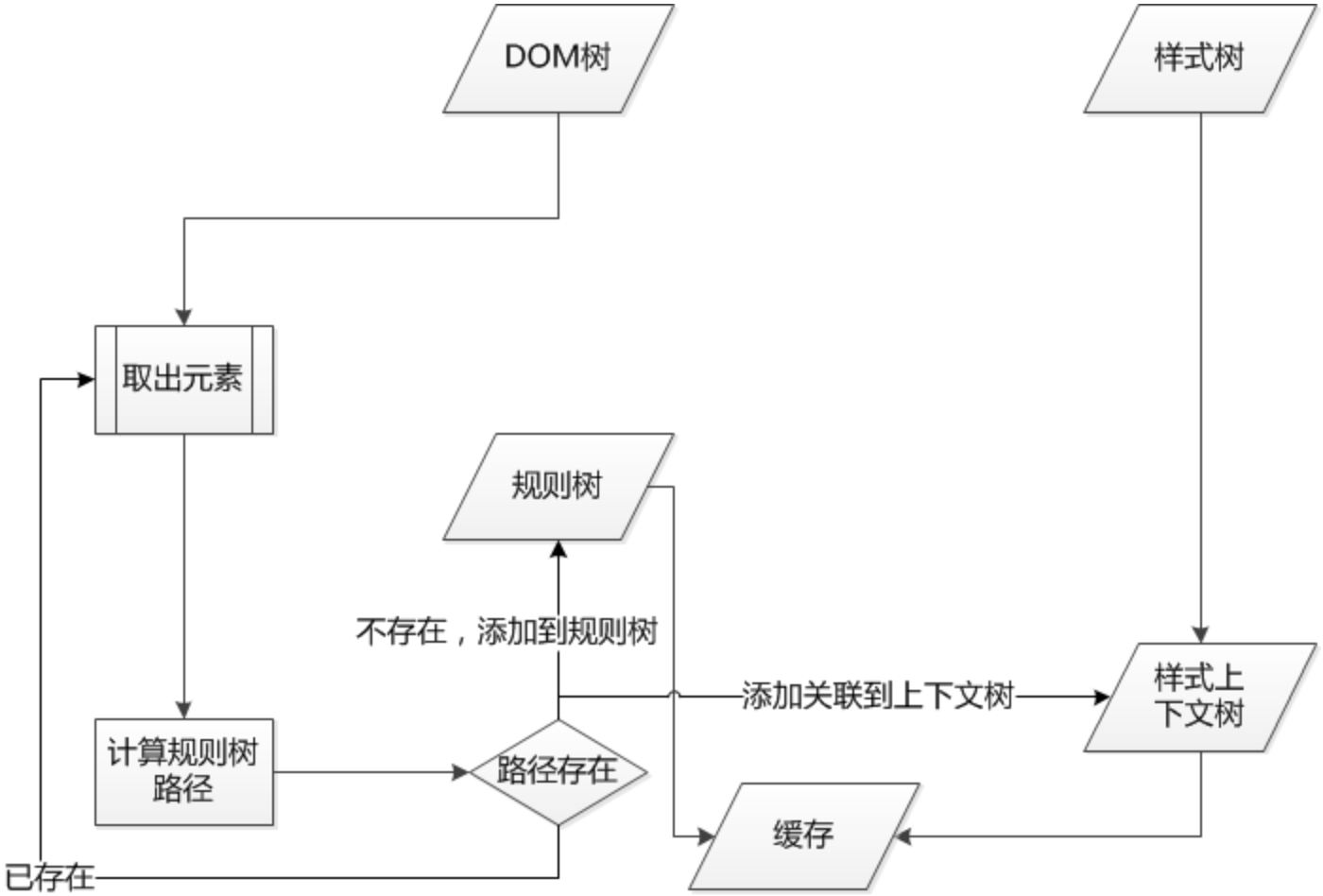
例子：映射



例子：映射







财付通设计中心
Tenpay Innovation Design

webkit - 共享样式数据

Webkit - 共享样式数据

- webkit节点引用样式对象（渲染样式），某些情况下，这些对象可以被节点间共享，这些节点需要是兄弟或是表兄弟节点，并且：
  - 这些元素必须处于相同的鼠标状态（比如不能一个处于hover，而另一个不是）
  - 不能有元素具有id
  - 标签名必须匹配
  - class属性必须匹配
  - 对应的属性必须相同
  - 链接状态必须匹配
  - 焦点状态必须匹配
  - 不能有元素被属性选择器影响
  - 元素不能有行内样式属性
  - 不能有生效的兄弟选择器，webcore在任何兄弟选择器相遇时只是简单的抛出一个全局转换，并且在它们显示时使整个文档的样式共享失效，这些包括+选择器和类似:first-child和:last-child这样的选择器。

遍历匹配

- Webkit中，并没有规则树，匹配的声明会被遍历四次
  - 先是应用非important的高优先级属性（之所以先应用这些属性，是因为其他的依赖于它们 - 比如display），
  - 其次是高优先级important的，
  - 接着是一般优先级非important的，
  - 最后是一般优先级important的规则。

  这样，出现多次的属性将被按照正确的**级联顺序**进行处理，最后一个生效。

样式表的级联顺序

- 一个样式属性的声明可能在几个样式表中出现，或是在一个样式表中出现多次，因此，应用规则的顺序至关重要，这个顺序就是级联顺序。
- 根据css2的规范，级联顺序为（从低到高）：
  - 1. 浏览器声明
  - 2. 用户声明
  - 3. 作者的一般声明
  - 4. 作者的important声明
  - 5. 用户important声明

Specifity（权值）

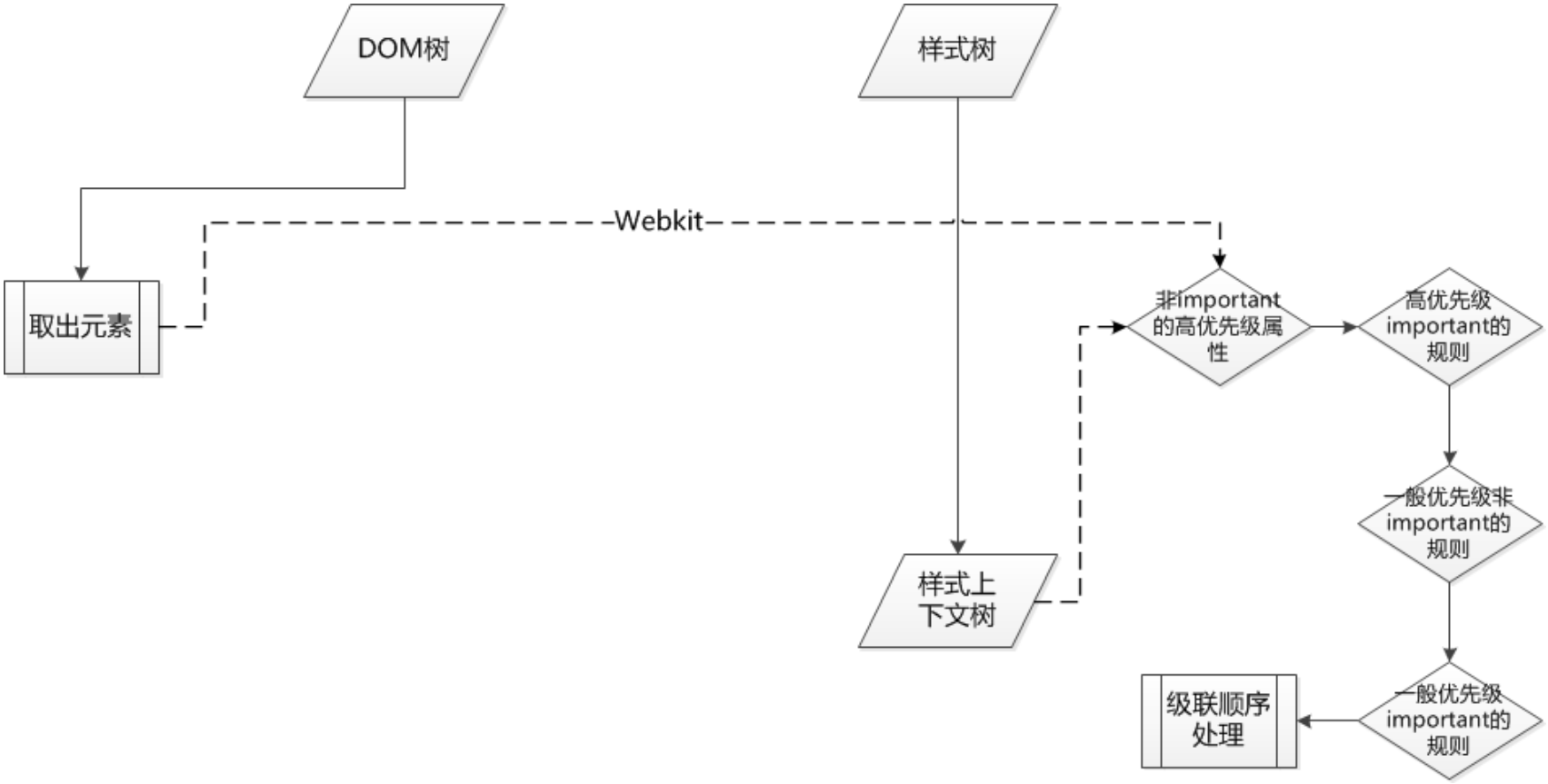
- Css2规范中定义的选择符specifity如下：
  - 如果声明来自style属性，而不是一个选择器的规则，则计1，否则计0（＝a）
  - 计算选择器中id属性的数量（＝b）
  - 计算选择器中class及伪类的数量（＝c）
  - 计算选择器中元素名及伪元素的数量（＝d）
- 连接a - b - c - d四个数量（用一个大基数的计算系统）将得到specifity。这里使用的**基数由分类中最高的基数定义**。

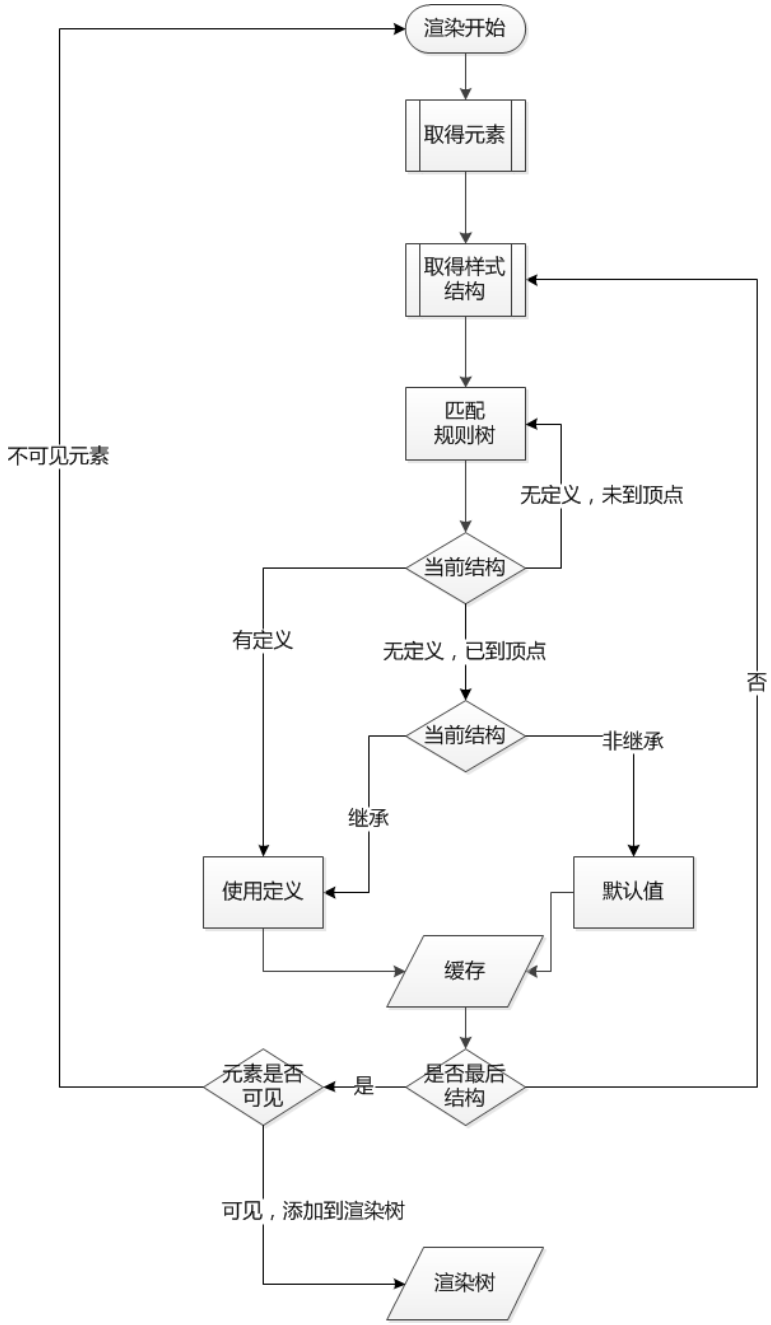
规则排序

- 规则匹配后，需要根据级联顺序对规则进行排序，webkit先将小列表用冒泡排序，再将它们合并为一个大列表。

逐步处理

- webkit使用一个标志位标识所有顶层样式表都已加载，如果在attch时样式没有完全加载，则放置占位符，并在文档中标记，一旦样式表完成加载就重新进行计算。


DOM树
样式树
Webkit
取出元素
非important的高优先级属性
高优先级important的规则
一般优先级非important的规则
样式上下文树
级联顺序处理
一般优先级important的规则



财付通设计中心
Tenpay Innovation Design

布局

layout/reflow

布局

- 当渲染对象被创建并添加到树中，它们并没有位置和大小，计算这些值的过程称为layout或reflow。
- Html使用基于流的布局模型。流中靠后的元素并不会影响前面元素的几何特性，所以布局可以在文档中从右向左、自上而下的进行。也存在一些例外，比如html tables。
- 坐标系统相对于根frame，使用top和left坐标。
- 根渲染对象的位置是0,0，它的大小是viewport - 浏览器窗口的可见部分。
- 布局是一个递归的过程，由根渲染对象开始，它对应html文档元素，布局继续递归的通过一些或所有的frame层级，为每个需要几何信息的渲染对象进行计算。

Dirty bit 系统

- 为了不因为每个小变化都全部重新布局，浏览器使用一个dirty bit系统，一个渲染对象发生了变化或是被添加了，就标记它及它的children为dirty。
- 存在两个标识
  - dirty：需要layout。
  - children are dirty：即使这个渲染对象可能没问题，但它至少有一个child需要layout。

全局和增量 layout

- 当layout在整棵渲染树触发时，称为全局layout，这可能在下面这些情况下发生：
  - 1. 一个全局的样式改变影响所有的渲染对象，比如字号的改变
  - 2. 窗口resize
- layout也可以是增量的，这样只有标志为dirty的渲染对象会重新布局（也将导致一些额外的布局）。增量 layout会在渲染对象dirty时异步触发，例如，当网络接收到新的内容并添加到Dom树后，新的渲染对象会添加到渲染树中。

异步和同步layout

- 增量layout的过程是异步的
  - Firefox为增量layout生成了reflow队列，以及一个调度执行这些批处理命令。
  - Webkit也有一个计时器用来执行增量layout - 遍历树，为dirty状态的渲染对象重新布局。
- 另外，当脚本请求样式信息时，例如“offsetHeight”，会同步的触发增量布局。
- 全局的layout一般都是同步触发。
- 有些时候，layout会被作为一个初始layout之后的回调，比如滑动条的滑动。

优化

- 当一个layout因为resize或是渲染位置改变（并不是大小改变）而触发时，渲染对象的大小将会从缓存中读取，而不会重新计算。
- 一般情况下，如果只有子树发生改变，则layout并不从根开始。这种情况发生在，变化发生在元素自身并且不影响它周围元素，例如，将文本插入文本域（否则，每次击键都将触发从根开始的重排）。

layout过程

layout过程

1. parent渲染对象决定它的宽度
2. parent渲染对象读取chilidren，并：
   a. 放置child渲染对象（设置它的x和y）
   b. 在需要时（它们当前为dirty或是处于全局layout或者其他原因）调用child渲染对象的layout，这将计算child的高度
3. parent渲染对象使用child渲染对象的累积高度，以及margin和padding的高度来设置自己的高度 - 这将被parent渲染对象的parent使用
4. 将dirty标识设置为false

- Firefox使用一个“state”对象（nsHTMLReflowState）做为参数去布局（firefox称为reflow），state包含parent的宽度及其他内容。
- Firefox布局的输出是一个“metrics”对象（nsHTMLReflowMetrics）。它包括渲染对象计算出的高度。

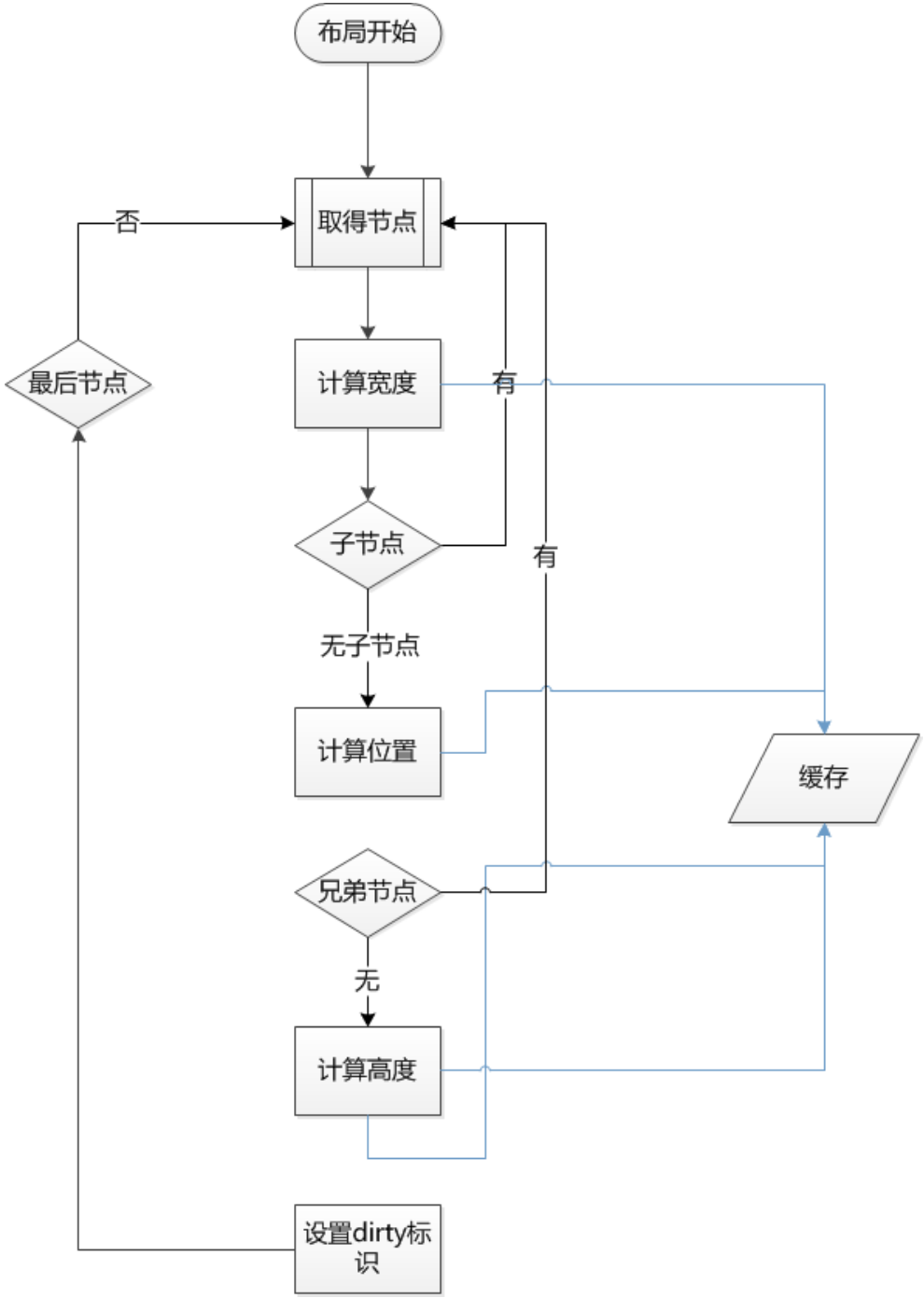
宽度计算

- webkit中宽度的计算过程是（RenderBox类的calcWidth方法）：
  - 容器的宽度是容器的可用宽度和0中的最大值，这里的可用宽度为：contentWidth=clientWidth()-paddingLeft()-paddingRight()
    - clientWidth和clientHeight代表一个对象内部的不包括border和滑动条的大小
  - 元素的宽度指样式属性width的值，它可以通过计算容器的百分比得到一个绝对值
  - 加上水平方向上的border和padding
- 到这里是最佳宽度的计算过程，现在计算宽度的最大值和最小值
  - 如果最佳宽度大于最大宽度则使用最大宽度
  - 如果小于最小宽度则使用最小宽度。

  最后缓存这个值，当需要layout但宽度未改变时使用。

Line breaking

- 当一个渲染对象在布局过程中需要折行时，则暂停并告诉它的parent它需要折行，parent将创建额外的渲染对象并调用它们的layout。



财付通设计中心
Tenpay Innovation Design

绘制

显示内容

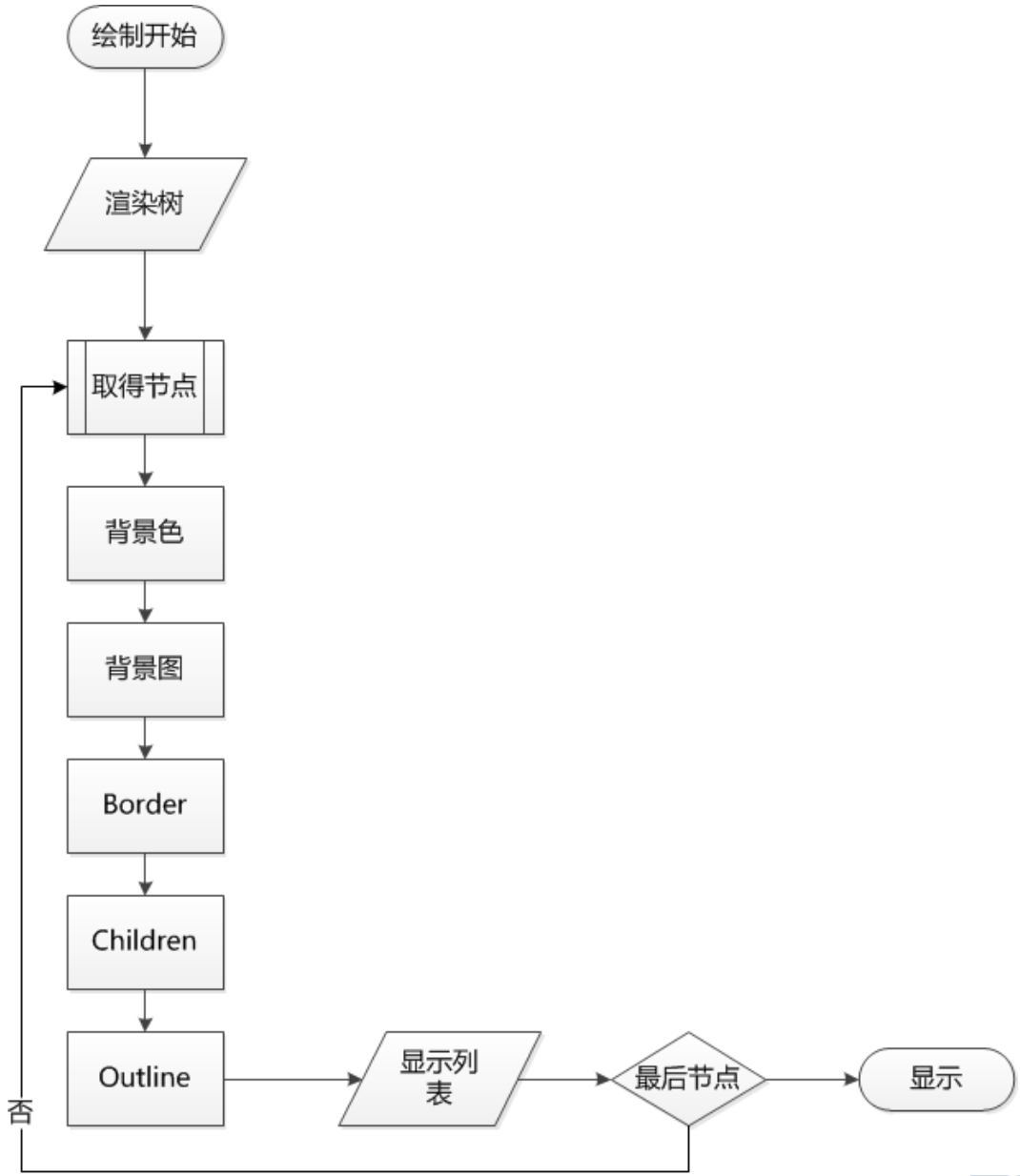
全局和增量

- 和布局一样，绘制也可以是全局的 - 绘制完整的树 - 或增量的。
  - 在增量的绘制过程中，一些渲染对象以不影响整棵树的方式改变，改变的渲染对象使其在屏幕上的矩形区域失效，这将导致操作系统将其看作dirty区域，并产生一个paint事件，操作系统很巧妙的处理这个过程，并将多个区域合并为一个。Chrome中，这个过程更复杂些，因为渲染对象在不同的进程中，而不是在主进程中。Chrome在一定程度上模拟操作系统的行为，表现为监听事件并派发消息给渲染根，在树中查找到相关的渲染对象，重绘这个对象（往往还包括它的children）。

绘制顺序

- 这个就是元素压入堆栈的顺序，这个顺序影响着绘制，堆栈从后向前进行绘制。
- 一个块渲染对象的堆栈顺序是：
  1. 背景色
  2. 背景图
  3. border
  4. children
  5. outline

css2定义了绘制过程的顺序：http://www.w3.org/TR/CSS21/zindex.html



财付通设计中心
Tenpay Innovation Design

总结

工作中需要注意的内容

代码

- 不规范的HTML结构会降低页面解析效率（HTML）
- HTML的节点、层级越少页面解析效率越高（HTML）
- 通配选择符对性能的影响几乎可以忽略（CSS）
- 包含选择符的层级过多会导致降低样式解析效率（CSS）
- 不显示的对象不会被渲染（CSS）

动态变化

- 浏览器总是试着以最小的动作响应一个变化
  - 一个元素颜色的变化将只导致该元素的重绘
  - 元素位置的变化将导致元素的布局和重绘
  - 添加一个Dom节点，也会大致这个元素的布局和重绘
  - 一些主要的变化，比如增加html元素的字号，将会导致缓存失效，从而引起整数的布局和重绘。

Thanks

no Q&A

CSSForest.org